



Distributed Systems

Design Issues

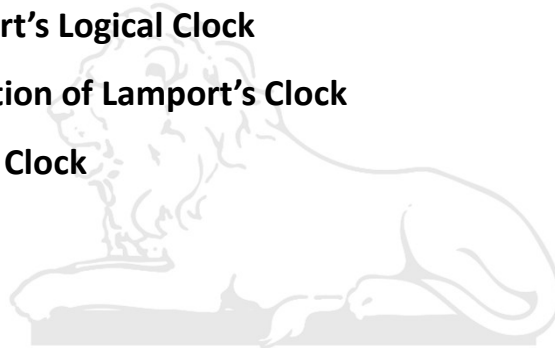
Issue 1: Time in Distributed Systems



Recap



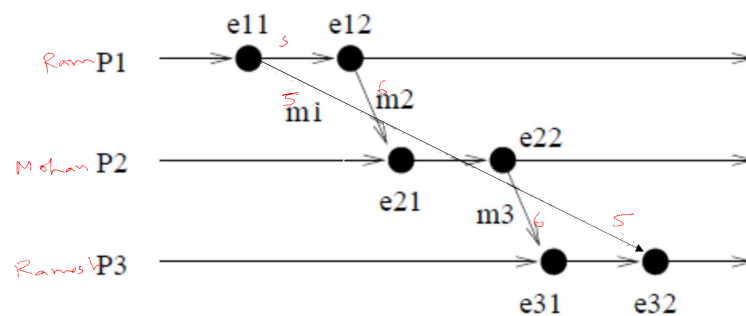
- Lamport's Logical Clock
- Limitation of Lamport's Clock
- Vector Clock



Application of vector clocks: Causal ordering of messages:

- If $\text{send}(m_1) \rightarrow \text{send}(m_2)$, then every recipient of both message m_1 and m_2 must “deliver” m_1 before m_2 .
 - “deliver” – when the message is actually given to the application for processing
- Causal ordering of messages is needed for applications such as **replicated database systems**

Violation of Causal Message Ordering



$m_1 \rightarrow m_2 \rightarrow m_3 \Rightarrow m_1 \rightarrow m_3$

Birman-Schiper-Stephenson Protocol

- Implemented in ISIS wherein processes are assumed to communicate using **broadcast messages**
- Requires that messages should be delivered reliably

BASIC IDEA

- Deliver a message to a process only if the message immediately preceding it has been delivered to the process.
- Each message has a timestamp that is used to check this.

Ref: Singhal M & Shivaratri N, Advanced Concepts in OS

ISIS (1987)



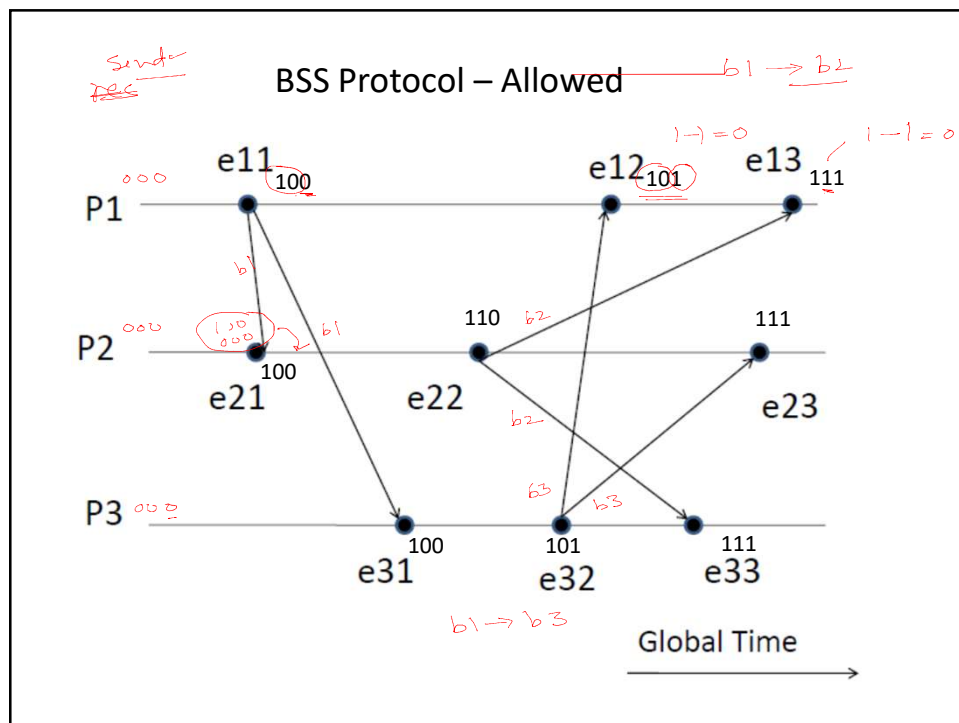
- A system for fault-tolerant Distributed Computing (Toolkit)
- Birman founded Isis Distributed Systems to commercialize this software
- The software was used by
 - New York and Swiss Stock Exchanges for more than a decade
 - French air traffic control system and
 - the US Navy AEGIS warship
- Later evolved in Vsync library

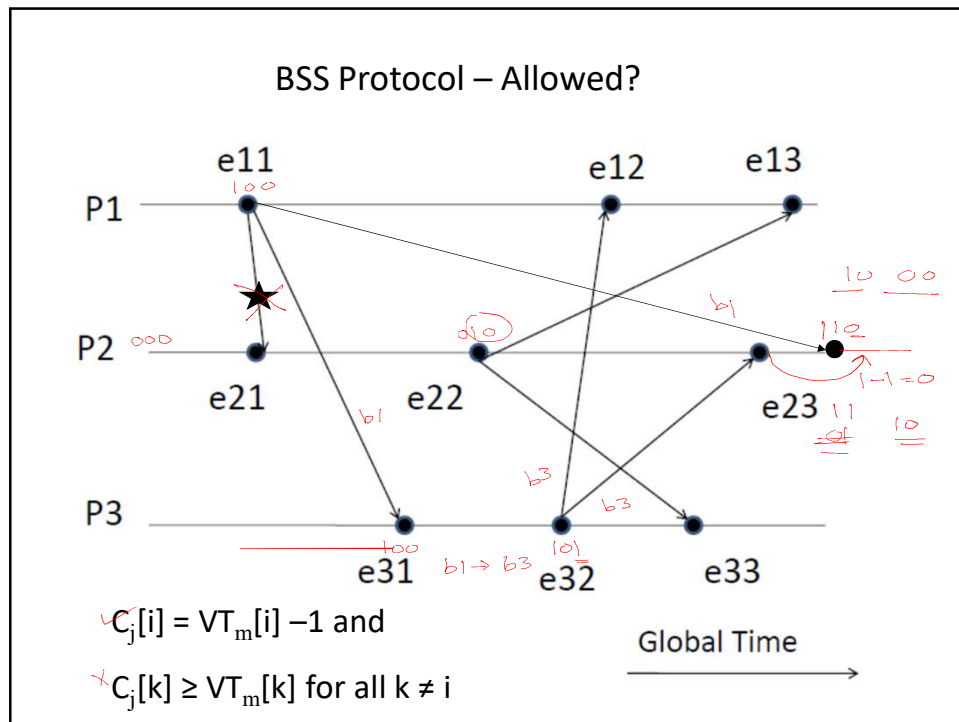
Wikipedia (25-01-2025)

IIT ROORKEE

Birman-Schiper-Stephenson Protocol

- To broadcast m from process i , increment $C_i[i]$, and timestamp m with $VT_m = C_i(a)$
- When $j \neq i$ receives m , j delays delivery of m until
 - $C_j[i] = VT_m[i] - 1$ and
 - $C_j[k] \geq VT_m[k]$ for all $k \neq i$
 - **Delayed messages are queued in j sorted by vector time. Concurrent messages are sorted by receive time**
- When m is delivered at j , C_j is updated according to vector clock rule 2
- Protocol is deadlock free.

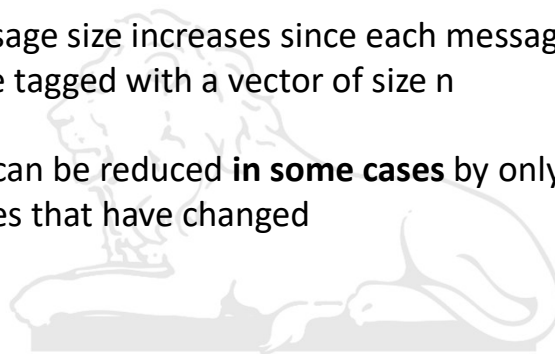


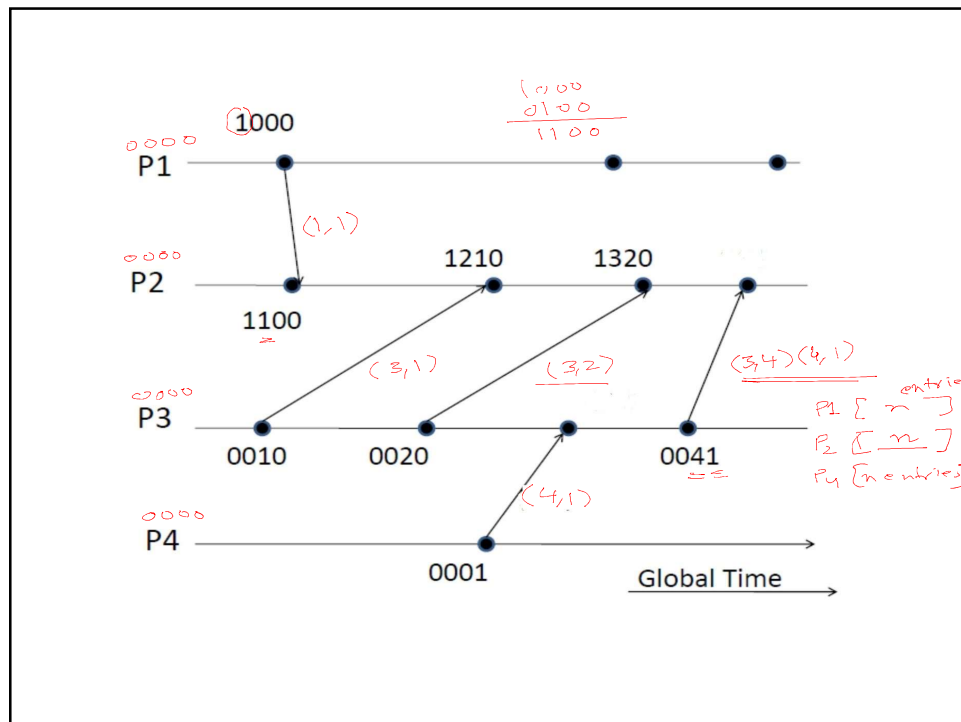


Problem of Vector Clock



- message size increases since each message needs to be tagged with a vector of size n
- size can be reduced **in some cases** by only sending values that have changed





An Efficient Implementation of Vector Clocks

Singhal and A. Kshemkalyani

Observation

When the number of processes are high only few processes are likely to frequently interact

=> **Between two successive sending events from process P_i to process P_j , only a few of the vector clock's entries are expected to change.**

- Piggyback only the information relative to the entries that changed.
- Send pairs (PID, Vector clock of PID)

An Efficient Implementation of Vector Clocks

- When P_j receives a message with compressed timestamp

$$[(i_1, v_1), (i_2, v_2), \dots, (i_{n1}, v_{n1})] \quad \langle \underline{3}, 2 \rangle \langle \underline{4}, 1 \rangle \leftarrow$$

it updates its vector clock as:

$$\underline{vt_j[i_k]} = \max(vt_j[i_k], v_k) \quad \text{for all } k \text{ in message}$$

- Each process to remember the vector timestamp in the message last sent to every other process
- Storage overhead (direct implementation) – $O(n^2)$
- Requires FIFO channels